MINI PROJECT REPORT

EVENTNEST

A Modern Component-Driven React 19 Frontend Integrated with a Layered Express REST API and PostgreSQL Relational Storage

Submitted by:

Aadi Charan

University Roll No: AU2024BCSE109

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

(Kalvium Work-Integrated Learning Program)

Course: Full Stack Web Development

Under the guidance of

Faculty Mentor

Associate Professor, ASAC



ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY, BENGALURU

ACADEMIC YEAR: 2026–2027

COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the micro-project report titled **"EventNest: College Event Registration Platform"** submitted by **Aadi Charan [Uni. Roll No. AU2024BCSE109]** in partial fulfilment for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (Kalvium Work-Integrated Learning Program)** for the course **Full Stack Web Development** at Alliance University, Bengaluru, is a bonafide record of original work accomplished under my supervision and guidance during the academic year **2026–2027**.

This report embodies the genuine results of design, implementation, and empirical verification conducted by the student, and its contents have not formed the basis for the award of any other degree, diploma, or certificate to the candidate or any other person.

Aadi Charan	**Faculty Mentor**	**Head of Department**
Student Candidate	Associate Professor, ASAC	School of Advanced Computing

COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the micro-project report titled **"EVENTNEST: College Event Registration Platform"**, submitted by me in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (CSE)** for the course **Full Stack Web Development** at Alliance University, is a genuine record of my own academic work carried out under the guidance of my faculty mentor.

I further confirm that all explanations, architectural descriptions, database designs, API endpoints, and empirical test outcomes presented in this document represent the actual implementation of the EventNest codebase. All technical references and conceptual foundations have been acknowledged properly in the bibliography.

This declaration stands as a testament to the authenticity, academic integrity, and personal accountability of the work submitted.

Date: 24 October 2026
Place: Bengaluru, Karnataka

Aadi Charan
University Reg. No: AU2024BCSE109

Table of Contents

1. Introduction & Full-Stack System Context 4
2. Full-Stack System Architecture & Data Flow 5
3. Frontend Technology Stack & Tooling 5
4. Component Hierarchy & User Interface Structure 6
5. Client-Side Routing & Navigation Workflows 7
6. State Management, React Hooks & Form Handling 8
7. Centralized API Service Layer & Error Handling 9
8. Genuine Application Evidence & User Interface Walkthrough 10
9. Limitations, Future Enhancements & Conclusion 12

1. Introduction & Full-Stack System Context

In a typical college campus environment, students participate in hackathons, seminars, robotics competitions, and technical workshops. Traditional event sign-up mechanisms, such as static paper forms or disconnected Google Forms, lack live seating constraints, instantaneous validation, and real-time database synchronization. As a consequence, students often arrive at workshops only to find that all seats have been claimed, or administrators find duplicate registrations cluttering attendance lists.

EventNest was engineered as a comprehensive full-stack solution to these operational challenges. Rather than relying on monolithic, server-rendered HTML pages, EventNest adopts a modern **decoupled Single-Page Application (SPA) architecture**. The frontend is built using React 19 and Vite, providing a fast, component-driven user interface. The backend is an Express REST API that communicates over HTTP and JSON, backed by a persistent PostgreSQL database.

1.1 Core Full-Stack Objectives

- **Decoupled Architecture:** Completely separate presentation logic (React) from business logic and database persistence (Express & PostgreSQL).
- **Seamless Single-Page Navigation:** Enable instant transitions between the catalog, event details, registration forms, and attendance records without full browser reloads using client-side routing.
- **Defensive Client-Side Validation:** Provide immediate visual feedback when form fields are empty or invalid using React Hook Form, reducing unnecessary network calls.
- **Real-Time Seat Synchronisation:** Reflect seat counts and "Event Full" badges dynamically across all views as new registrations are recorded in the database.

> **Full-Stack Philosophy:** The frontend does not dictate database rules; it mirrors backend constraints. Validation occurs on both tiers—on the frontend for instant user experience, and on the backend for absolute data integrity.

2. Full-Stack System Architecture & Data Flow

EventNest follows a **client-server architecture** connected by a standardized RESTful HTTP interface.

2.1 End-to-End Data Flow

1. **User Interaction (Browser):** The student interacts with React components running locally on `http://localhost:5173`.
2. **API Service Layer (`api.js`):** Client actions trigger asynchronous JavaScript `fetch()` requests to endpoints like `/api/events` or `/api/registrations`.
3. **Vite Development Proxy:** In development, Vite intercepts requests targeting `/api/*` and transparently proxies them to `http://localhost:5000`, eliminating cross-origin browser issues while maintaining relative URLs.
4. **Express REST API:** The Express server processes the request, applies middleware (CORS, JSON body parser), runs controller validation, and communicates with PostgreSQL.
5. **PostgreSQL Database:** PostgreSQL executes parameterized SQL queries and returns tabular result sets to Node.js.
6. **Reactive UI Re-render:** The backend sends JSON responses back to the React app, which updates component state and updates the Virtual DOM.

3. Frontend Technology Stack & Tooling

The frontend is built with modern, industry-standard web technologies:

- **React 19 (v19.2):** Utilizes functional components, React Hooks (`useState`, `useEffect`), and efficient Virtual DOM diffing for rapid UI updates.
- **Vite 8 (v8.3):** A next-generation frontend build tool providing Hot Module Replacement (HMR) during development and optimized, minified production bundling.
- **React Router DOM (v7.18):** Provides client-side declarative routing, URL parameter binding (`/events/:id`), and query string synchronization.
- **React Hook Form (v7.89):** Manages form state, performs input validation with customizable rules, and manages submission lifecycles with minimal re-renders.
- **Vanilla CSS Design System:** Clean, responsive CSS with CSS custom properties (variables), subtle shadows, card layouts, and status badge color accents.

4. Component Hierarchy & User Interface Structure

EventNest follows a modular, atomic component hierarchy that separates reusable UI elements from top-level page views.

4.1 Component Architecture Map

```
frontend/src/
├── main.jsx                # Application entry point, renders <App /> into #root
├── App.jsx                 # Root router, global layout, Navbar, and Footer
├── index.css               # Global design tokens, typography, grid, and button styles
├── components/
│   ├── Navbar.jsx          # Global header with brand logo and navigation links
│   ├── EventCard.jsx       # Reusable card displaying event info and seat availability
│   └── Alert.jsx           # Reusable dismissible status banner (success, error, info)
├── pages/
│   ├── Home.jsx            # Landing page with hero banner, quick actions, and features
│   ├── Events.jsx          # Campus event catalog with grid layout and search overview
│   ├── EventDetails.jsx    # Deep-dive view for a single event with registration CTA
│   ├── CreateEvent.jsx     # Administrative form for publishing new campus events
│   ├── Register.jsx        # Student registration form with event selector and feedback
│   └── Registrations.jsx   # Historical roster table displaying all student registrations
└── services/
    └── api.js              # Centralized fetch abstraction communicating with backend
```

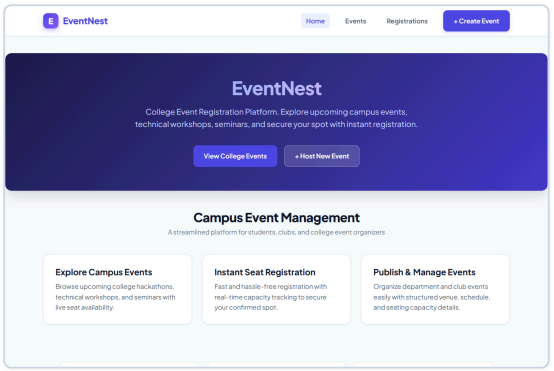


Figure 4.1: The EventNest Home Dashboard (Home.jsx) presenting the hero banner, quick navigation cards, and live system highlights.

4.2 Reusable Component: `EventCard.jsx`

The `EventCard` component encapsulates event presentation across both the home and catalog views. It visually indicates seat availability with color-coded badges (green for available, red for full), formats dates, and renders conditional action buttons:

```
// frontend/src/components/EventCard.jsx (Excerpt)
export default function EventCard({ event }) {
  const isFull = event.is_full || event.available_seats <= 0;
  return (
    <div className="event-card">
      <div className="event-card-header">
        <span className={`badge ${isFull ? 'badge-full' : 'badge-available'}`}>
          {isFull ? 'Full' : `${event.available_seats} Seats Available`}
        </span>
        <h3>{event.title}</h3>
      </div>
      <p className="event-desc">{event.description}</p>
      <div className="event-meta">
        <span>🗓️ {event.date} at {event.time}</span>
        <span>📍 {event.venue}</span>
      </div>
      <div className="event-card-actions">
        <Link to={`/events/${event.id}`} className="btn btn-secondary">Details</Link>
        {!isFull && <Link to={`/register?eventId=${event.id}`} className="btn btn-primary">Register</Link>}
      </div>
    </div>
  );
}
```

5. Client-Side Routing & Navigation Workflows

Traditional multi-page web applications reload the entire HTML document whenever the user navigates between views. In contrast, EventNest utilizes **React Router v7** to deliver instant, smooth client-side routing.

5.1 Declarative Route Configuration in `App.jsx`

The top-level route tree maps URL paths to page components:

```
// frontend/src/App.jsx: Declarative routing configuration
export default function App() {
  return (
    <Router>
      <Navbar />
      <main className="main-content">
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/events" element={<Events />} />
          <Route path="/events/:id" element={<EventDetails />} />
          <Route path="/register" element={<Register />} />
          <Route path="/create-event" element={<CreateEvent />} />
          <Route path="/registrations" element={<Registrations />} />
          <Route path="*" element={<Navigate to="/" replace />} />
        </Routes>
      </main>
      <footer className="footer">...</footer>
    </Router>
  );
}
```

5.2 URL Query String Synchronization for Event Selection

When a student browses the catalog and clicks "Register" on an event card, the app navigates to `/register?eventId=3`. Inside `Register.jsx`, the `useSearchParams` hook reads the query parameter and automatically pre-selects that event in the dropdown:

```
// frontend/src/pages/Register.jsx
const [searchParams] = useSearchParams();
const preselectedEventId = searchParams.get('eventId');

useEffect(() => {
  if (preselectedEventId) {
    setValue('event_id', String(preselectedEventId), {
      shouldValidate: true,
      shouldDirty: true,
    });
  }
}, [preselectedEventId, setValue]);
```

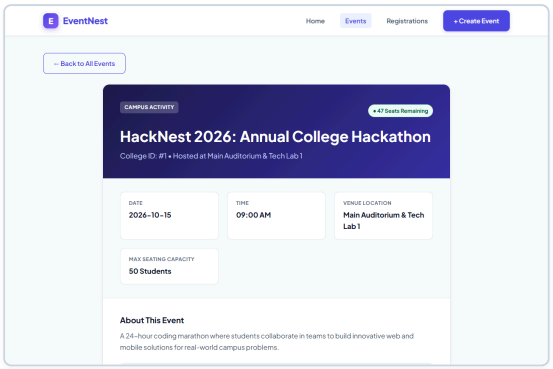


Figure 5.1: The Event Details page (/events/1) providing comprehensive event descriptions and direct registration linking.

6. State Management, React Hooks & Form Handling

State management in EventNest is designed to be lightweight, predictable, and local to the components that require it, avoiding the overhead of heavy third-party state libraries like Redux.

6.1 React Hooks in Action

- `useState`: Holds asynchronous data (events array, registration rosters), loading flags (`loading` boolean), and error states (`submitError` string).
- `useEffect`: Triggers asynchronous API calls immediately after a component mounts in the DOM, populating initial view state cleanly.
- `useNavigate`: Provides programmatic navigation (e.g., redirecting users after creating an event).

6.2 Robust Form Handling with React Hook Form

Both `CreateEvent.jsx` and `Register.jsx` employ **React Hook Form** to manage input validation and submission states. Validation rules are declared directly alongside input elements:

```
// Form validation rules in Register.jsx
<input
  type="text"
  placeholder="e.g. Rahul Sharma"
  {...register('student_name', {
    required: 'Student full name is required.',
    minLength: { value: 2, message: 'Name must be at least 2 characters.' },
  })}
/>
{errors.student_name && <span className="error-text">{errors.student_name.message}</span>}

<input
  type="email"
  placeholder="e.g. rahul.sharma@college.edu"
  {...register('student_email', {
    required: 'Email address is required.',
    pattern: {
      value: /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
      message: 'Please enter a valid college email address.',
    },
  })}
/>
```

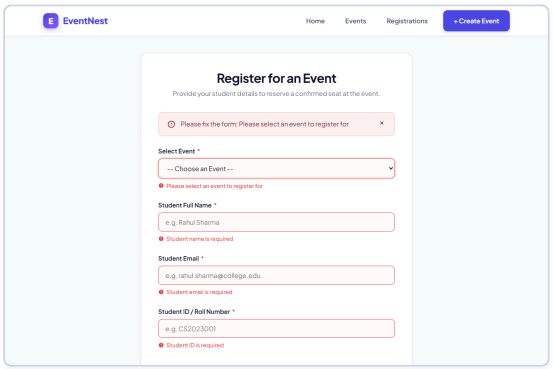


Figure 6.1: Immediate client-side validation triggered when attempting to submit an incomplete registration form.

7. Centralized API Service Layer & Error Handling

A key best practice in full-stack architecture is isolating network calls within a dedicated service layer rather than scattering raw `fetch()` invocations across UI components.

7.1 Centralized Service in `frontend/src/services/api.js`

All communication with the Express backend is routed through `api.js`. The service normalizes JSON responses, handles HTTP error status codes consistently, and surfaces clear error messages:

```
// frontend/src/services/api.js: Centralized response handler
async function handleResponse(response) {
  const isJson = response.headers.get('content-type')?.includes('application/json');
  const data = isJson ? await response.json() : null;

  if (!response.ok) {
    const error = (data && data.message) || `HTTP error! Status: ${response.status}`;
    const errObj = new Error(error);
    errObj.status = response.status;
    errObj.data = data;
    throw errObj;
  }

  return data;
}

export async function getEvents() {
  const response = await fetch('/api/events');
  return handleResponse(response);
}

export async function registerStudent(regData) {
  const response = await fetch('/api/registrations', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(regData),
  });
  return handleResponse(response);
}
```

7.2 Defensive User Feedback via `Alert.jsx`

Whenever an API request succeeds or fails, the frontend presents clean, dismissible notification banners rendered by `components/Alert.jsx`:

- **Success Banners (Green):** Display confirmation messages upon successful registration or event publication.
- **Error Banners (Red):** Display backend validation errors, duplicate registration warnings (HTTP 409), or capacity limit alerts (HTTP 400).

8. Genuine Application Evidence & User Interface Walkthrough

The following genuine screenshots demonstrate the live EventNest full-stack application running across real browser sessions on `http://localhost:5173` backed by live Express and PostgreSQL instances.

8.1 Campus Events Catalog

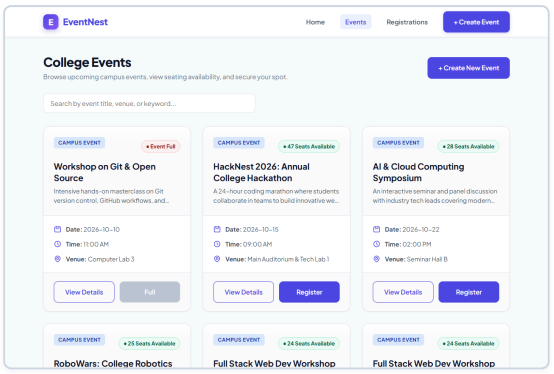


Figure 8.1: Events Catalog view displaying chronological campus events with dynamic seat capacity badges.

8.2 Create Campus Event Form

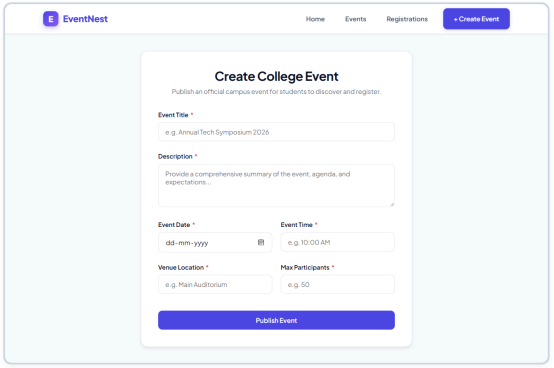


Figure 8.2: Administrative event creation interface validating title, description, venue, and participant capacity limits.

8.3 Edge Case Handling: Duplicate Registrations (409 Conflict)

When a student attempts to register for an event they have already signed up for, the backend rejects the request with HTTP 409 Conflict, and the frontend displays a clear warning:

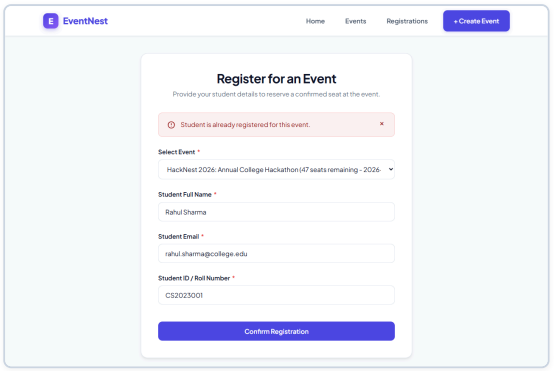


Figure 8.3: Instant visual feedback when duplicate registration is detected and blocked.

8.4 Edge Case Handling: Full Event Capacity Rejection (400 Bad Request)

When a user selects an event that has reached maximum capacity, the submission is rejected with "Event is full.":

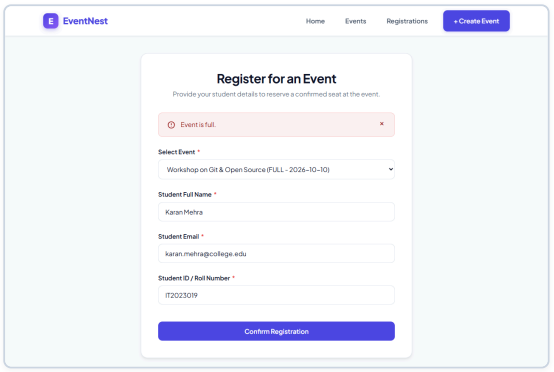


Figure 8.4: Defensive capacity enforcement preventing registration when maximum participant limit is reached.

8.5 Successful Registration & Summary Card

Upon successful registration, the form clears and renders a registration summary card confirming the registered student and event title:

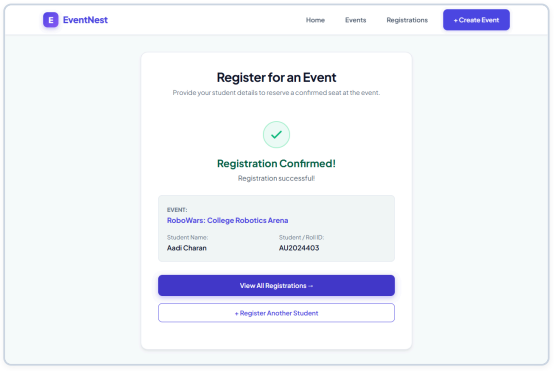


Figure 8.5: Confirmation summary card displayed after successful registration insertion.

8.6 Complete Registrations Roster

The Registrations page (`/registrations`) fetches all joined student and event records from PostgreSQL and renders a chronological tabular roster:

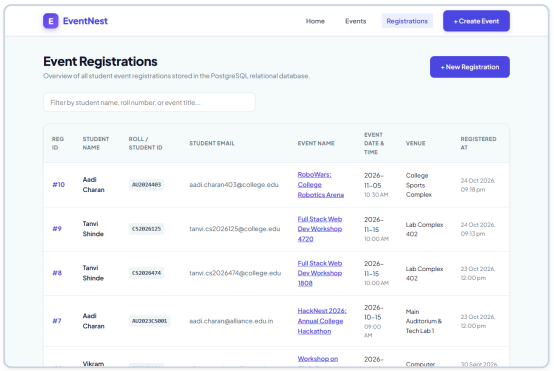


Figure 8.6: Live registrations overview table displaying student details, roll numbers, timestamps, and registered event names.

9. Limitations, Future Enhancements & Conclusion

9.1 Limitations

- **No Student Login:** Any visitor can sign up without verifying a password or university SSO.
- **No Search Bar / Category Filter:** Events are displayed chronologically; filtering by tech category or search keyword is not yet implemented.
- **Client-Side Polling:** Seat capacity is refreshed on page navigation rather than via real-time WebSockets.

9.2 Future Enhancements

- **JWT Authentication:** Add protected student and admin profiles with university email verification.
- **QR Code Ticket Generation:** Generate downloadable QR passes for check-in at event venues.
- **Dark Mode & Theme Toggles:** Provide CSS custom property toggles for accessible dark mode viewing.

9.3 Conclusion

EventNest successfully demonstrates full-stack web development principles by connecting a reactive, user-friendly React 19 interface to an Express REST API and PostgreSQL database. The application enforces validation on both tiers, handles edge cases gracefully, and provides an engaging, responsive experience for campus event management.

10. References

1. React Core Team, *React 19 Documentation & Best Practices*, https://react.dev
2. Vite Team, *Vite Build Tool Next Generation Guide*, https://vite.dev
3. React Router Community, *React Router v7 Documentation*, https://reactrouter.com
4. React Hook Form Contributors, *React Hook Form API Reference*, https://react-hook-form.com
5. MDN Web Docs, *Single Page Application (SPA) Architectural Patterns*, https://developer.mozilla.org